



Nombre y Apellido: _____

Legajo: _____

Examen Parcial 2

Observación: entregar la resolución de cada ejercicio en hojas separadas.

1. Definimos el TAD *List*, de listas finitas, con las siguientes operaciones:

tad *List* (*A* : Ordered Set) where

import Bool

nil : *List A* -- construye una lista vacía

cons : *A* → *List A* → *List A* -- agrega un elemento a la lista

elem : *A* → *List A* → Bool -- determina si un elemento está en la lista

remove : *A* → *List A* → *List A* -- elimina la ocurrencia de un elemento en la lista

removeDups : *List A* → *List A* -- remueve los elementos duplicados en una lista $[1,1,2,2] \rightarrow [1,2]$

sort : *List A* → *List A* -- ordena los elementos de una lista $[1,3,2,3,2,4] \rightarrow [1,2,2,3,3,4]$

- a) Dar una especificación algebraica del TAD.

- b) Dadas las siguientes definiciones sobre listas en Haskell:

sorted :: Ord a ⇒ [a] → Bool

sorted (x : y : xs) = x ≤ y ∧ *sorted* (y : xs)

sorted ys = True

maximum :: Ord a ⇒ [a] → a

maximum [x] = x

maximum (x : y : xs) | x ≤ y = *maximum* (y : xs)
 | True = *maximum* (x : xs)

last :: [a] → a

last [x] = x

last (x : y : xs) = *last* (y : xs)

Probar por inducción estructural sobre listas la siguiente propiedad:

$$xs \neq [] \wedge \text{sorted } xs \Rightarrow \text{maximum } xs = \text{last } xs$$

2. Se representan secuencias mediante árboles binarios dados por el siguiente tipo de datos:

data Tree a = E | Leaf a | Node (Tree a) (Tree a)

donde el recorrido *inorder* del árbol da el orden de los elementos de la secuencia.

Definir en Haskell de manera eficiente la función *intercalate* :: Tree a → Tree (Tree a) → Tree a, que dada una secuencia *s* y una secuencia de secuencias *xs*, inserta *s* entre los elementos de *xs* y aplana la secuencia resultado. Por ejemplo,

intercalate(0,1)([2,3],[4,5,6],[8]) = [2,3,0,1,4,5,6,0,1,8]

intercalate()([1,2],[3,4,5]) = [1,2,3,4,5]

intercalate(0,1)() = []

Definir *intercalate* con profundidad en $O(h)$, donde *h* es la altura del árbol que recibe como segundo argumento la función.

3. Dados los diferentes valores de las acciones de YPF a lo largo del tiempo (que se representan como una secuencia de valores que representan el valor de las acciones cada día), se desea saber cuál es la mejor ganancia que se puede obtener al comprar acciones un día y venderlas otro día posterior. Por ejemplo:

```

48: 6 7 8 9 10
mejorGanancia(10, 7, 5, 8, 11, 9) = 6
mejorGanancia(10, 9, 8, 7) = 0
mejorGanancia(12) = 0

```

donde en la primer secuencia la mejor ganancia 6 se obtiene comprando a 5 y vendiendo a 11, en la segunda secuencia la mejor ganancia es 0 porque la mejor opción es no comprar ni vender, y en la tercera es 0 porque no hay suficientes días para comprar y vender.

Utilizando las operaciones del TAD secuencias, completar la definición de la función `mejorGanancia : Seq Int → Int` que dada una secuencia de valores calcula la mejor ganancia que se puede obtener, dando definiciones para `reverse`, `op` y `b` de manera que se verifique profundidad en $O(\lg n)$ y trabajo en $O(n)$, siendo n el largo de la secuencia.

```

mejorGanancia : Seq Int → Int
mejorGanancia s =
  let n = length s
      mejorVenta = reverse (fst (scan op b (reverse s)))
      diff = tabulate (\i → (nth i mejorVenta) - nth i s) (n - 1)
  in if n < 2 then 0 else reduce max 0 diff

```